Cloud Computing (CS3081)

Name : Aniket Raj

Roll No. : 230103030

Github : https://github.com/iamankoo/cloud-assignment

1. Fundamental Learning & Conceptual Takeaways

Undertaking this mini project provided me with an end to end practical understanding of the interaction between ML systems and cloud infrastructure. It helped me to see how a machine learning model can be taken from a local development environment to a scalable, cloud based deployment where it can be accessed as a real service by users.

Rather than treating model training and service deployment as isolated tasks, this project demonstrated the complete lifecycle of an AI system in a cloud environment. It gave me hands-on experience in model development, containerization, deployment, and service management, while also highlighting key cloud concepts such as scalability, high availability, fault tolerance and cost awareness.

Overall, the project helped me build a clearer and more practical understanding of how cloud computing enables the reliable, efficient and large-scale serving of machine learning models.

(a) Sequence level knowledge distillation on consumer hardware, including its purpose, workflow and practical benefits such as model compression and faster inference.

(b) Cloud Native storage Decoupling & Containers Immutability, and how they help in building portable, consistent and easy-to-deploy ML services.

(c) Systems Automation & Stateless Network Security, including the use of automation (e.g., CI/CD) and best practices for securing cloud based services.

Scanned with OKEN Scanner

2. Technical Challenges Encountered & Engineering Solutions.

During the implementation of this project, I faced several real-world technical challenges related to model serving, storage management and container orchestration in a cloud environment. The key challenges and the corresponding solutions are described below.

Challenge 1 : Ollama cross-containers Network Refusal in k3d cluster.

Solution : I reconfigured and launched the host Ollama service with `OLLAMA_HOST = 0.0.0.0:11434`, enabled `OLLAMA_FLASH_ATTENTION = true`, and set `OLLAMA_NUM_PARALLEL = 2`, enabling multiple worker pods to query concurrent inference slots without starvation. Additionally, I updated the kubernetes service and network policies to allow stable and secure communication between the pods and the Ollama service.

Challenge 2 : SeaweedFS containers OOM Termination during model uploads.

Solution : I updated the `k8s/data/seaweedfs.yaml` to provision a 2 GiB persistentVolumeClaim (seaweedfs.pvc) mounted under /data and raised the memory limit to 1.5 GiB. This change provided enough storage and memory headroom for handling large model files, permanently resolving the storage persistence and memory starvation issues. I also monitored the container resource usage to ensure that the limits are optimal for stable operation.

Scanned with OKEN Scanner

3. Empirical Findings, SRE Observations & What could be done better.

After deploying and testing the system in the cloud environment, I collected several empirical results and SRE (Site Reliability Engineering) observations to evaluate the performance, reliability and cost efficiency of the setup. The key findings are summarised below.

Key Empirical Findings & SRE Observations :

1) Chaos Recovery & SRE Observation : Deleting an active pod under load resulted in recovery within 18.93 s, with zero dropped connections. The traffic was quickly and evenly distributed across the available replicas (53.4%, 46.6%), which shows that the system maintained high availability and handled the failure gracefully without any visible service interruption to the users.

2) Economic serving cost : Under the project rate card, the dual replica deployment costs 7.9 INR/hr. Serving 2052 requests/hr resulted in a unit cost of 3.95 INR per 1000 requests. This indicates that the setup is cost-effective for the achieved throughput and can be further optimised with auto-scaling based on real-time load.

What could have been done better :

1) Pre-hydrated persistent volumes : Using a shared ReadWriteMany volume with safetensors memory mapping (mmap = True) would eliminate network weight download and reduce the cold start time from 18.0 s to sub-second levels, leading to faster recovery and better user experience.

2) Stateful eBPF security policies : Replacing stateless NACLs with cilium eBPF network policies would allow connection tracking, and identity-aware filtering, while also giving finer control over allowed traffic without the need to manage ephemeral port return rules.

Overall, the observations show that the system is reliable, cost-efficient and scalable, and the above improvements can make it more robust and production-ready.

Scanned with OKEN Scanner